Lab 4: MATLAB for Mechatronics

Vectors, Plots, Filtering, Calibration, and Decisions

ENGR 1105 | Fall 2026

Name	**Partner**
Date	**Section**

Learning Objectives

By the end of this lab, you should be able to:

1. create and inspect MATLAB vectors;
2. calculate summary statistics;
3. write and run a MATLAB script;
4. create a labeled engineering plot;
5. import sensor data from a CSV file;
6. identify valid measurements using logical indexing;
7. apply a moving-average filter;
8. create a linear sensor calibration; and
9. classify measurements as danger, warning, or safe.

1. Lab Organization

Part	Main work
Lab 4.1	MATLAB interface, vectors, indexing, summary statistics, scripts, and engineering plots
Lab 4.2	CSV import, data validation, filtering, calibration, state classification, and results communication

2. Files

Keep the following files in the same MATLAB Current Folder:

File	Purpose
`lab4_starter.m`	Starter MATLAB script containing the required sections
`sample_sensor_data.csv`	Example ultrasonic distance data and reference values

Before Starting

Do not rename the CSV file unless you also update the file name inside the MATLAB script. Save your completed script using a new file name that includes your name.

Lab 4.1

MATLAB Vectors, Scripts, and Plots

Building a Repeatable Engineering Calculation

3. Part A: Set Up the MATLAB Folder

Instructions

1. Create one folder for Lab 4.
2. Place `lab4_starter.m` and `sample_sensor_data.csv` in that folder.
3. Open MATLAB.
4. Set the Lab 4 folder as the Current Folder.
5. Open `lab4_starter.m` in the Editor.
6. Save a copy using your name, such as `lab4_phan.m`.

MATLAB Environment

What is the full Current Folder path?

What variables appear in the Workspace after you run the first script section?

4. Part B: Create and Inspect Vectors

The starter script contains:

```
time = [0 0.2 0.4 0.6 0.8];
distance = [20.8 20.1 21.0 19.7 20.4];
```

Add commands that calculate:

- minimum distance;
- maximum distance;
- average distance;
- standard deviation;

- number of measurements.

Useful functions:

`min()`, `max()`, `mean()`, `std()`, `length()`.

Vector Results

Quantity	MATLAB result
Minimum distance	
Maximum distance	
Average distance	
Standard deviation	
Number of measurements	

5. Part C: Practice Indexing

Use MATLAB indexing to create the following variables:

```
firstDistance = distance(1);
thirdDistance = distance(3);
middleDistances = distance(2:4);
lastDistance = distance(end);
```

Then create a two-column matrix:

```
dataMatrix = [time' distance'];
```

The transpose operator converts each row vector into a column vector.

Indexing Results

Expression	Result
`distance(1)`	
`distance(3)`	
`distance(2:4)`	
`distance(end)`	
Size of `dataMatrix`	

What does `dataMatrix(:,2)` select?

6. Part D: Create an Engineering Plot

Add the following structure to the script and complete the missing information:

```
figure
plot(time,distance,"o-","LineWidth",1.5)
grid on
xlabel("_________________")
ylabel("_________________")
title("_________________")
```

The plot must include:

- time on the horizontal axis;
- distance on the vertical axis;
- units on both axis labels;
- markers and a connecting line;
- a descriptive title;
- a grid.

Checkpoint 1: MATLAB Foundations

Demonstrate the completed vector calculations, indexing commands, and labeled plot. Explain the difference between a vector and a single scalar value.

Plot Interpretation

At what time does the minimum distance occur?

Describe the main variation visible in the plot.

7. Part E: Element-by-Element Calculation

Suppose the distance values are measured in centimeters. Convert the entire vector to meters:

```
distanceM = distance / 100;
```

Now calculate squared distance using an element-by-element power:

```
distanceSquared = distanceM.^2;
```

Matrix Power and Element Power

Use `.^` when raising each vector element to a power. The operator `^` performs a matrix-power operation.

Element-by-Element Results

What is the first value of `distanceM`?

What is the first value of `distanceSquared`?

Why is the dot required in `distanceM.^2`?

8. Part F: Create a Simple Decision

Add:

```
currentDistance = distance(end);

if currentDistance < 20
    state = "danger";
elseif currentDistance < 21
    state = "warning";
else
    state = "safe";
end
```

Decision Result

What value is stored in `currentDistance`?

Which state is selected?

Change the final distance value to 19.5 and rerun the section. Which state is selected?

Lab 4.2

Sensor Data Analysis

Validation, Filtering, Calibration, and States

9. Part G: Import the CSV File

Run:

```
data = readmatrix( ...
    "sample_sensor_data.csv", ...
    "NumHeaderLines",1);

time = data(:,1);
rawDistance = data(:,2);
referenceDistance = data(:,3);
```

Imported Data

How many rows of numerical data were imported?

What does each column represent?

Column	Meaning
1	
2	
3	

10. Part H: Plot Raw and Reference Data

```
figure
plot(time,rawDistance,"o-")
hold on
plot(time,referenceDistance,"--", ...
    "LineWidth",1.5)
hold off

grid on
xlabel("Time (s)")
ylabel("Distance (cm)")
```

```
title("Raw and Reference Distance")
legend("Raw sensor","Reference", ...
    "Location","best")
```

Raw Data Interpretation

At which three approximate reference distances were data collected?

Does the raw sensor generally read above or below the reference value?

Checkpoint 2: Imported Sensor Data

Demonstrate the successful import and the labeled raw/reference plot. Explain what each column represents and why the original data should be preserved.

11. Part I: Validate the Measurements

Create a logical vector:

```
valid = rawDistance >= 2 ...
    & rawDistance <= 400;
```

Count valid rows:

```
validCount = sum(valid);
invalidCount = sum(~valid);
```

Create validated vectors:

```
validTime = time(valid);
validRawDistance = rawDistance(valid);
validReference = referenceDistance(valid);
```

Validation Results

Number of valid measurements: _____

Number of invalid measurements: _____

What does `~valid` mean?

_____

_____

12. Part J: Apply a Moving Average

```
filteredDistance = movmean( ...
    validRawDistance,3);
```

Create a plot containing raw, filtered, and reference values.

```
figure
plot(validTime,validRawDistance,"o-")
hold on
plot(validTime,filteredDistance, ...
    "LineWidth",1.5)
plot(validTime,validReference,"--", ...
    "LineWidth",1.5)
hold off

grid on
xlabel("Time (s)")
ylabel("Distance (cm)")
title("Raw, Filtered, and Reference Data")
legend("Raw","3-point moving average", ...
    "Reference","Location","best")
```

Filtering

Does the filtered data look smoother than the raw data?

What happens near the transitions from approximately 20 cm to 35 cm and from 35 cm to 50 cm?

Why can a larger moving-average window make a controller respond more slowly?

13. Part K: Calibrate the Sensor

Fit a line relating raw sensor distance to reference distance:

```
p = polyfit( ...
    validRawDistance, ...
    validReference,1);

scale = p(1);
offset = p(2);

calibratedDistance = polyval( ...
    p,validRawDistance);
```

The fitted model has the form

$$d_{\text{calibrated}} = m d_{\text{raw}} + b.$$

Calibration Model

Scale factor, m: _____

Offset, b: _____

Write the completed calibration equation:

_____

_____

14. Part L: Compare Errors

```
rawError = validRawDistance ...
    - validReference;

calibratedError = calibratedDistance ...
    - validReference;

meanAbsRawError = mean(abs(rawError));

meanAbsCalibratedError = ...
    mean(abs(calibratedError));
```

Error Comparison

Mean absolute raw error: _____

Mean absolute calibrated error: _____

Did the calibration improve the measurement? Support your answer using the calculated errors.

_____

_____

Checkpoint 3: Filtering and Calibration

Demonstrate the moving-average plot and calibration model. Explain the filtering tradeoff and whether calibration reduced the mean absolute error.

15. Part M: Classify Operating States

Use calibrated distance:

```
danger = calibratedDistance < 25;

warning = calibratedDistance >= 25 ...
    & calibratedDistance < 40;

safe = calibratedDistance >= 40;
```

Create state labels:

```
state = strings(size(calibratedDistance));

state(danger) = "danger";
state(warning) = "warning";
state(safe) = "safe";
```

Count each state:

```
dangerCount = sum(danger);
warningCount = sum(warning);
safeCount = sum(safe);
```

State Results

State	Count	Distance rule
Danger		
Warning		
Safe		

Why must the three logical rules cover every valid distance without overlapping incorrectly?

16. Part N: Create a Results Table

```
results = table( ...
    validTime, ...
```

```
    validRawDistance, ...
    filteredDistance, ...
    calibratedDistance, ...
    state);

disp(results)
```

Checkpoint 4: Final Analysis

Demonstrate the final results table, state counts, and one final labeled plot. Explain the complete process from imported raw data to an engineering decision.

17. Troubleshooting

Problem	Check first
MATLAB cannot find the CSV file	Verify the Current Folder, file name, and extension.
Undefined variable	Run the earlier script section that creates the variable.
Plot command gives a dimension error	Confirm that the horizontal and vertical vectors have equal lengths.
Matrix dimensions do not agree	Check whether element-by-element operators such as `.*`, `./`, or `.^` are required.
Index exceeds array bounds	Check the vector size and remember that MATLAB indexing begins at 1.
Calibration is unreasonable	Check units, imported columns, reference values, and whether the fit used valid data only.
State array contains empty entries	Check that the danger, warning, and safe conditions cover all valid values.

18. Part O: Design Challenge

Design Challenge

Choose one challenge:

1. **Filter comparison:** Compare moving-average windows of 3, 5, and 7. Plot all three and explain the tradeoff.
2. **Error plot:** Plot raw and calibrated error versus time. Add a horizontal line at zero error.
3. **Custom warning zones:** Create four states: critical, danger, warning, and safe. Choose and justify the boundaries.
4. **Your Week 3 data:** Replace the sample data with measurements collected during Lab 3 and repeat the analysis.

Design Record

Which challenge did you choose?

Describe the MATLAB commands or logic you added.

What did the resulting plot or table show?

What limitation remains in your analysis?

19. Final Reflection

Explain how MATLAB supports the sense-decide-act model even though MATLAB is not directly controlling the LED or motor during this lab.

Explain the difference among raw, filtered, and calibrated data.

20. Submission Checklist

☐ Completed Lab 4 instruction sheet

☐ Completed MATLAB script

☐ Vector calculations and indexing results

☐ Raw/reference and raw/filtered/reference plots

☐ Calibration equation and error comparison

☐ Final state table and counts

☐ Completed design challenge and reflection